OpenPA: An Open-Source Personal Assistant-as-a-Service with LLM-Driven Tool Orchestration and Autonomous Code Generation

Maxwell Au

Matriculation Number: U2321963E

College of Computer and Data Science
Nanyang Technological University

Course Project Report

April 2026

Source code: https://github.com/maxwellau2/OpenPA

Contents

1 Introduction **3**

2 Literature Review **4**

2.1 Commercial Personal Assistants 4
2.2 Open-Source Agent Frameworks 5
2.3 Coding Agents 5
2.4 Retrieval-Augmented Generation and Personalisation 6
2.5 Tool Calling Protocols 6
2.6 Prompt Engineering Techniques 6

3 Research Gap **7**

4 Problem Statement **8**

5 Solution Design **9**

5.1 System Architecture Overview 9
5.2 MCP-Based Tool Architecture (P1) 10
5.3 Adaptive Agent Loop (P2) 11
5.3.1 Mode A: Plan-Then-Execute 11
5.3.2 Mode B: Adaptive Loop 11
5.3.3 Tool Filtering (P5) 12
5.3.4 Error Recovery 12
5.4 RAG-Based Personalisation (P3) 12
5.4.1 Architecture 13
5.4.2 Memory Lifecycle 13
5.5 Vibe-Coding Workspace (P4) 13
5.5.1 Tool Suite 13
5.5.2 Adaptive Workflow 14
5.5.3 Code Inspection 14
5.6 Self-Evolution: The Assistant That Extends Itself 14
5.6.1 Evolution Triggers 15
5.6.2 Safety Bounds 16
5.7 System Prompt Design 16
5.8 Multi-Provider LLM Support (P5) 17

6 Illustrative Examples **17**

6.1 Example 1: Daily Briefing (Multi-Service Orchestration) 17
6.2 Example 2: Semantic Memory and RAG (Personalisation) 18
6.3 Example 3: Vibe-Coding (Autonomous Code Generation) 18

6.4 Example 4: Cross-Service Workflow (Plan-Then-Execute) 19

7 Evaluation **20**

7.1 Qualitative Comparison with Baselines 20

7.2 Service Integration Scope 21

7.3 Context-Aware Tool Filtering 22

7.4 Multi-Provider LLM Compatibility 22

7.5 Agent Execution Characteristics 23

8 Conclusion **24**

8.1 Summary 24

8.2 Limitations 24

8.3 Future Work 24

8.4 Closing Remarks 26

1 Introduction

The modern digital professional interacts with a fragmented ecosystem of services daily—email, version control, messaging platforms, music streaming, social media, calendars, and RSS feeds, among others. Each service operates in isolation with its own interface, notification system, and workflow. Commercial personal assistants such as Apple Siri, Google Assistant, and Amazon Alexa have attempted to unify this experience, but remain constrained by vendor lock-in, limited third-party integrations, and an inability to perform complex, multi-step tasks autonomously.

Recent advances in large language models (LLMs) have introduced a new paradigm: *agentic AI systems* capable of reasoning over tasks, invoking external tools, and adapting their behaviour based on context [1, 2]. Projects such as Claude Code [3], OpenAI's Codex CLI [4], and the Pi coding agent [5] demonstrate that LLMs can autonomously read, write, and test code. However, these systems are standalone coding tools—they do not integrate with the broader digital life of a user.

This project presents **OpenPA** (*Open Personal Assistant*), an open-source, self-hosted Personal Assistant-as-a-Service platform that addresses this gap. OpenPA orchestrates 16 heterogeneous services—including Gmail, GitHub, Spotify, Discord, Telegram, Mastodon, Google Calendar, RSS, and YouTube—through a unified natural language chat interface powered by LLMs. The system is built on Anthropic's Model Context Protocol (MCP) [6], which provides a standardised interface for tool discovery and invocation.

OpenPA makes the following contributions:

1. **Unified multi-service orchestration**: 88 MCP-compliant tools across 16 services, accessible through natural language with automatic credential management and per-user isolation.

2. **RAG-based personalisation**: A retrieval-augmented generation system using locally-hosted embeddings (Ollama `nomic-embed-text`) and ChromaDB [7] that learns about the user over time and injects relevant context into every conversation.

3. **Autonomous code generation (vibe-coding)**: A workspace-based development environment with 15 tools for cloning repositories, editing files, running tests, and submitting pull requests—with an adaptive test-fix loop inspired by Claude Code's architecture [3].

4. **Self-evolving architecture**: Because OpenPA's own source code is hosted on GitHub, the vibe-coding system can modify the assistant's own codebase. Users can request new integrations, tools, or features in natural language (e.g., "add a LinkedIn tool to OpenPA"), and the agent will read existing code, generate the new tool following established patterns, write tests, verify them, and submit a pull request—enabling the platform to evolve autonomously.

5. **Multi-provider LLM support with BYOK**: A provider-agnostic Bring-Your-Own-Key (BYOK) architecture supporting 6 LLM backends (Anthropic Claude, Google Gemini, OpenAI, Groq, OpenRouter, and Ollama for local models). The server holds no provider credentials—each user supplies their own API keys, ensuring cost transparency and eliminating vendor dependency.

6. **Context-aware tool filtering**: A dynamic tool selection mechanism that reduces the tool set exposed to the LLM from 88 to as few as 8 tools based on task context, enabling effective use of smaller local models.

The remainder of this report is structured as follows. Section 2 reviews related work in personal assistants, agentic AI, and coding agents. Section 3 identifies the research gap. Section 4 formalises the problem statement. Section 5 presents the solution design in detail. Section 6 provides illustrative examples of the system in operation. Section 7 evaluates the system through qualitative comparison with baselines and analysis of tool filtering, multi-provider compatibility, and agent execution characteristics. Section 8 concludes with a discussion of limitations and future work.

2 Literature Review

This section surveys related work across four domains: commercial personal assistants, open-source agent frameworks, coding agents, and retrieval-augmented generation for personalisation.

2.1 Commercial Personal Assistants

Commercial personal assistants—Apple Siri, Google Assistant, and Amazon Alexa—provide voice-driven interfaces to a curated set of services. Google Assistant integrates with Gmail, Calendar, and Google Home devices; Alexa extends to smart home control via "Skills." However, these systems share common limitations:

- **Vendor lock-in**: Each assistant operates within its parent ecosystem. Google Assistant cannot control Spotify playback on non-Google devices; Siri cannot interact with Discord or Mastodon.
- **Shallow tool use**: Tasks are limited to single-step commands ("set a timer," "play music"). Multi-step workflows requiring reasoning—such as "check my PRs, summarise them, and send the summary to my team on Discord"—are not supported.
- **No code generation**: None of these assistants can write, test, or deploy code.
- **No self-hosting**: Users cannot run these systems on their own infrastructure, raising privacy concerns for sensitive data such as email and messages.

2.2 Open-Source Agent Frameworks

The emergence of LLM-based agents has produced several open-source frameworks. LangChain [8] provides abstractions for chaining LLM calls with tool use, memory, and retrieval. AutoGPT [9] demonstrated autonomous goal-directed behaviour using GPT-4, iteratively breaking down objectives into sub-tasks. OpenClaw proposed multi-tool orchestration with a focus on composability.

These frameworks establish important architectural patterns—tool registries, agent loops, and memory systems—but they are *frameworks*, not end-user applications. They require significant development effort to integrate with specific services and lack built-in support for OAuth credential management, multi-tenancy, or user-facing chat interfaces.

2.3 Coding Agents

A distinct category of agentic systems focuses on autonomous code generation and modification.

Claude Code [3] is Anthropic's terminal-based coding agent. Its architecture features an async-generator-based agent loop with explicit state transitions, streaming tool execution with concurrency control, and a permission system with pattern matching. The system uses deferred tool loading to avoid overwhelming the model with the full tool set on every turn, and supports context compaction to manage long conversations within context window limits.

OpenAI Codex CLI [4] provides a terminal-based interface for code generation, execution, and file manipulation. It uses a simpler agent loop but demonstrates effective integration of sandboxed code execution with file system operations.

GitHub Copilot [10] operates as an inline code suggestion engine within IDEs. While highly effective for single-function completions, it is not an agentic system—it does not reason over multi-step tasks, invoke external tools, or manage project-level workflows.

Pi [5] is a minimal terminal coding harness with four core tools: `read`, `write`, `edit`, and `bash`. Its key architectural insight is that tool errors are returned as structured tool results rather than exceptions, allowing the LLM to read the error and recover autonomously. Pi also employs `beforeToolCall`/`afterToolCall` hooks for validation and supports extensibility through TypeScript extensions and prompt templates.

These coding agents are powerful but operate in isolation. None integrates with broader digital services (email, messaging, music), and none provides multi-user support or persistent personalisation.

2.4 Retrieval-Augmented Generation and Personalisation

Retrieval-Augmented Generation (RAG) [11] augments LLM generation with retrieved context from external knowledge stores. By embedding queries and documents into a shared vector space, RAG systems retrieve the most semantically relevant information and inject it into the model's context window.

RAG has been widely adopted for document question-answering, but its application to *user personalisation* in conversational agents remains underexplored. Most personal assistants rely on explicit user settings or session-level context; they do not maintain a persistent semantic memory that grows over time and retrieves relevant user facts based on the current conversation.

2.5 Tool Calling Protocols

Anthropic's Model Context Protocol (MCP) [6] provides a standardised protocol for connecting LLMs to external tools and data sources. MCP defines a client-server architecture where tool servers expose typed function signatures, and LLM clients discover and invoke these tools at runtime. This decouples the agent's reasoning from the tool implementations, enabling a plugin-like architecture.

2.6 Prompt Engineering Techniques

Effective prompting is critical for agent performance. Chain-of-thought prompting [12] enables step-by-step reasoning. The ReAct framework [1] interleaves reasoning traces with action execution. Analysis of production systems such as Claude Code [3] and Pi [5] reveals additional techniques:

- **Prompt sandwich**: Static system instructions encapsulate dynamic user context and tool definitions, with rules placed both before and after the variable content.
- **Tool preference hierarchy**: The system prompt explicitly instructs the model which tool to use for which task (e.g., "use `workspace_grep` for searching, not `bash` with `grep`").
- **Output efficiency directives**: Instructions to "lead with the answer, not the reasoning" and "skip filler words" reduce token waste and improve user experience.
- **Adaptive progress injection**: Periodic reminders of the original task and progress so far prevent model drift during long multi-turn sessions.

3 Research Gap

The literature reveals a fragmented landscape where different systems address different aspects of the personal assistant problem, but no existing solution combines all of them. Table 1 summarises the capabilities of existing systems against the requirements of a comprehensive personal assistant.

Table 1: Capability comparison of existing systems

System	Multi-Svc	Code Gen	Self-Evolve	RAG Mem.	Self-Host	Multi-LLM	Multi-User
Google Assistant	Partial	✗	✗	✗	✗	✗	✗
Siri / Alexa	Partial	✗	✗	✗	✗	✗	✗
Claude Code	✗	✓	✗	✗	✗	✗	✗
Codex CLI	✗	✓	✗	✗	✓	✗	✗
Pi Agent	✗	✓	✗	✗	✓	Partial	✗
LangChain/AutoGPT	✗*	✗	✗	Partial	✓	✓	✗
OpenPA	✓	✓	✓	✓	✓	✓	✓

*Frameworks, not end-user applications; require custom integration per service.

The following specific gaps are identified:

1. **No unified multi-service orchestration in open-source**: No existing open-source system provides integrated access to email, code hosting, messaging, music, social media, calendars, and RSS through a single agent. Users must switch between multiple applications and interfaces.

2. **Coding agents are isolated from digital life**: Claude Code, Codex CLI, and Pi are powerful coding tools, but they cannot send the results of their work to teammates on Discord, schedule follow-up tasks on a calendar, or notify stakeholders via email. The coding workflow is disconnected from the communication workflow.

3. **No self-evolving personal assistants**: No existing PA can modify its own codebase. When a user needs a new integration or feature, they must wait for a developer to implement it. A truly autonomous assistant should be able to extend its own capabilities—reading its source code, generating new tools following established patterns, testing the changes, and deploying them—all from a natural language request. This "self-evolution" capability would transform the PA from a static product into a living system that grows with its users' needs.

4. **Shallow personalisation**: Commercial assistants store explicit preferences ("preferred temperature unit") but do not build a semantic understanding of the user over time. No system uses RAG to retrieve personality traits, interests, and relationships by relevance to the current conversation.

5. **Vendor-locked LLM selection**: Most agents are tied to a single LLM provider. Users cannot choose between a free local model (Ollama) for casual tasks and a high-quality cloud model (Claude, GPT-4o) for complex tasks.

6. **Tool overload for small models**: When 80+ tools are exposed to a small local model (7–9B parameters), the model's context window is consumed by tool definitions, and tool-calling accuracy degrades. No existing open-source agent dynamically filters tools based on task context.

4 Problem Statement

Given the gaps identified in Section 3, we define the problem as follows:

> *How do we design and implement an open-source, self-hosted personal assistant platform that (1) orchestrates heterogeneous digital services through a unified natural language interface, (2) supports multiple LLM providers without vendor lock-in, (3) learns about users over time through semantic memory, (4) can autonomously write, test, and deploy code, and (5) scales to multiple users with per-user credential and data isolation?*

This problem decomposes into five sub-problems:

1. **Service Integration (P1)**: How do we provide a uniform interface to services with vastly different APIs (REST, OAuth, WebSocket, local CLI)? The system must handle authentication, rate limiting, and error recovery for each service without exposing this complexity to the user or the LLM.

2. **Agent Orchestration (P2)**: How does the LLM decide which tools to call, in what order, and how to handle failures? The agent must support both pre-planned multi-step workflows (e.g., "search the web, summarise, and send via Telegram") and adaptive loops where the next action depends on the result of the previous one (e.g., "run tests; if they fail, fix the code and re-test").

3. **Personalisation (P3)**: How does the system build a persistent understanding of each user that improves over time? The system must store observations about user preferences, personality, interests, and relationships, and retrieve only the *relevant* subset for each conversation—not all memories.

4. **Autonomous Code Generation (P4)**: How can the agent modify real codebases safely? It must clone repositories, understand existing code, make targeted changes, run the full test suite and build pipeline, fix any failures, and submit changes via pull request—all without human intervention.

5. **Model Compatibility (P5)**: How do we ensure the system works with both large cloud models (70B+ parameters, strong tool-calling) and small local models (7–9B parameters, limited context windows)? The system must adapt the information it sends to the model based on the model's capabilities.

5 Solution Design

This section presents the architecture of OpenPA, mapping each design decision to the sub-problems defined in Section 4.

5.1 System Architecture Overview

OpenPA follows a three-tier architecture:

- **Frontend**: A Next.js 16 [13] web application providing the chat interface, settings management, and real-time streaming via Server-Sent Events (SSE).
- **Backend**: A FastAPI [14] server hosting the agent loop, LLM provider abstraction, MCP tool registry, RAG system, and REST API.
- **Data Layer**: SQLite for relational data (users, conversations, credentials) and ChromaDB [7] for vector embeddings (semantic memory and conversation search).

Figure 1 illustrates the high-level architecture and data flow between components.

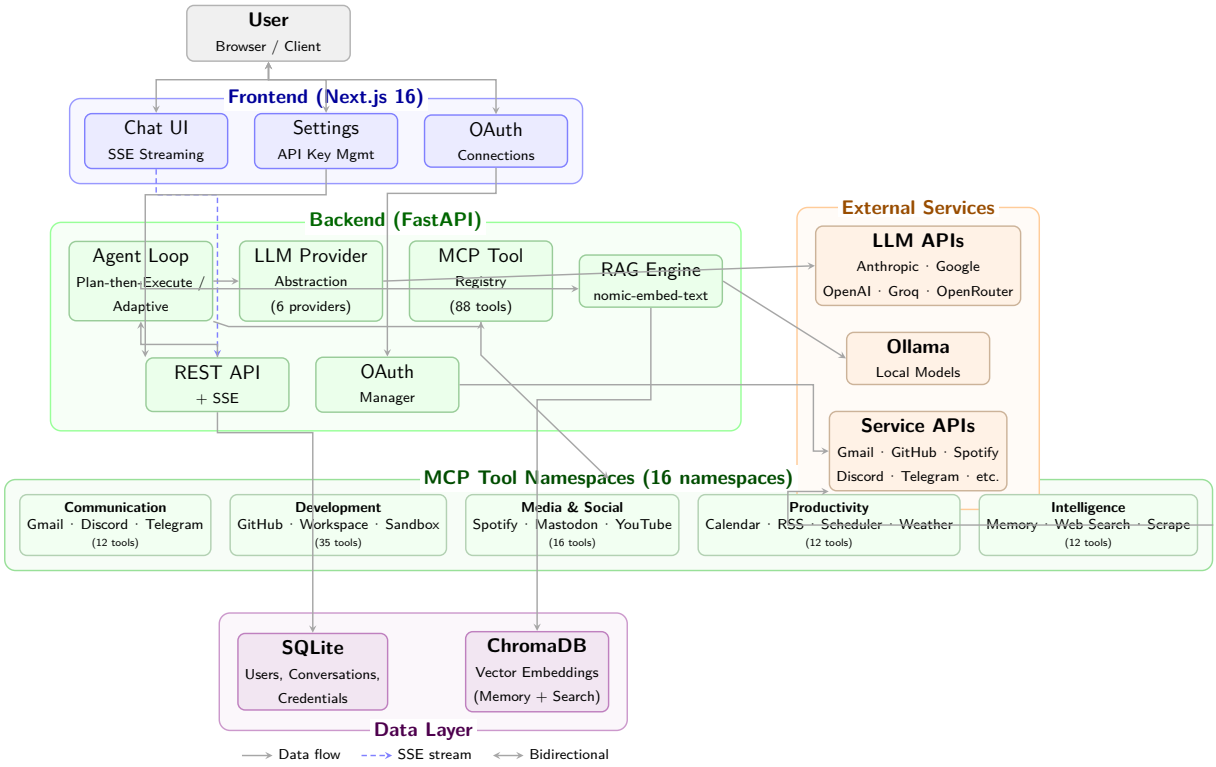

Figure 1: High-level architecture of the OpenPA system. The frontend communicates with the FastAPI backend via REST and SSE. The agent loop orchestrates LLM inference, tool execution across 16 MCP namespaces (88 tools), and RAG-based retrieval. Data persists in SQLite and ChromaDB, while external LLM and service APIs are accessed through provider abstractions and OAuth-managed credentials.

5.2 MCP-Based Tool Architecture (P1)

To address the service integration problem, OpenPA adopts Anthropic's Model Context Protocol (MCP) [6] as the universal tool interface. Each service is implemented as a self-contained FastMCP [15] server with typed tool definitions.

```python
@mcp.tool()
async def send_message(_user_id: int, to: str, message: str):
    """Send a Telegram message. Auto-resolves contact names."""
    creds = await get_creds(_user_id, "telegram")
    # ... send via Telegram API
```

Listing 1: Example tool definition (simplified)

All 16 service servers are mounted into a composite registry with namespace prefixes:

```python
mcp = FastMCP("openpa")
mcp.mount(gmail_mcp, namespace="gmail")      # gmail_send_email, ...
mcp.mount(github_mcp, namespace="github")    # github_create_pr, ...
mcp.mount(workspace_mcp, namespace="ws")     # ws_workspace_run, ...
# ... 12 more services
```

Listing 2: Tool registry mounting

This architecture yields 88 tools across 16 namespaces. Each tool receives a `_user_id` parameter injected by the agent loop, ensuring per-user credential isolation. OAuth tokens are stored per-user in the database and fetched on demand via a `get_creds(user_id, service)` helper.

Table 2 summarises the service integrations.

Table 2: OpenPA service integrations

Service	**Tools**	**Key Capabilities**
GitHub	13	Repos, PRs, issues, branches, file read/write, notifications
Workspace	15	Clone, edit, grep, inspect, run tests/builds, commit, push
Memory	8	Preferences, semantic memories, conversation search
Gmail	4	Unread, read, send, reply (with search syntax)
Mastodon	9	Timelines, trending, search, post, notifications
Spotify	5	Play, pause, search, playlists, current track
Discord	4	Servers, channels, send/read messages
Telegram	4	Contacts, send/read messages, chat listing
Calendar	3	List, create, delete events
Sandbox	7	Run Python/JS, multi-file test, verify syntax, export files
Scheduler	3	Schedule future tool calls, list, cancel
Web Scrape	3	Fetch page text, extract tables, extract links
RSS	5	Fetch, list, add, remove feeds
YouTube	2	Download video, get video info
Web Search	1	DuckDuckGo search
Weather	2	Current conditions, forecast
Total	**88**	

5.3 Adaptive Agent Loop (P2)

The agent loop is the core reasoning engine. OpenPA implements a dual-mode execution model:

5.3.1 Mode A: Plan-Then-Execute

For multi-step tasks that do *not* involve coding (e.g., "search the web for coffee roasters, compose a summary, and send it via Telegram"), the agent first generates an execution plan as a JSON array:

```
[
  {"step": 1, "tool": "web_search", "args": {"query": "..."}},
  {"step": 2, "tool": "llm_generate", "args": {"prompt": "Summarise..."
      }, "depends_on": 1},
  {"step": 3, "tool": "telegram_send_message", "args": {"to": "...", "
      content": ""}, "depends_on": 2}
]
```

Listing 3: Example execution plan

Steps linked by `depends_on` receive the output of the referenced step. A regex-based template replacement system handles various placeholder formats (e.g., `{{step_2.output}}`) that LLMs tend to generate, with a safety net that replaces any remaining `{{...}}` templates with the most recent step's result.

5.3.2 Mode B: Adaptive Loop

For coding tasks and simple queries, the agent uses an adaptive loop where the LLM decides the next action based on tool results. This is critical for vibe-coding, where test failures require reading the error, fixing the code, and re-testing—a flow that cannot be pre-planned.

```
while iterations < max_iterations:
    response = await provider.chat(messages, tools, system_prompt)
    if response.has_tool_calls:
        for tool_call in response.tool_calls:
            result = await call_tool(tool_call.name, tool_call.arguments
                )
            # Result appended to conversation; LLM sees it next
                iteration
    else:
        break  # LLM chose to respond with text; task complete
```

Listing 4: Simplified adaptive loop

The planner explicitly routes coding tasks to the adaptive loop:

> *"If the request involves CODING, WORKSPACE, or VIBE-CODING, ALWAYS output* `{"simple": true}`*. These tasks MUST use the adaptive loop so you can react to test failures."*

Workspace tasks receive an extended iteration limit (50 vs. 30) to accommodate multiple test-fix cycles. Progress reminders are injected every three iterations to prevent model drift.

5.3.3 Tool Filtering (P5)

To address the model compatibility problem, OpenPA implements context-aware tool filtering. Tools are categorised into nine semantic groups (core, workspace, github, communication, media, web, social, productivity, sandbox). Keywords in the user's message determine which categories are exposed:

```
# "Add a health check endpoint" -> workspace + github + core = 34 tools
# "Play some music"             -> media + core = 15 tools
# "Who am I?"                   -> core only = 8 tools
```

Listing 5: Tool filtering example

This reduces the tool set from 88 to as few as 8 tools, enabling smaller models (7–9B parameters) to function effectively. When workspace tools are called during execution, the filter dynamically expands to include GitHub and web tools for related operations.

5.3.4 Error Recovery

Inspired by the Pi coding agent [5], tool errors are returned as structured JSON messages rather than exceptions:

```
{"error": true, "tool": "github_create_pr",
 "message": "Validation Failed: base branch 'main' not found",
 "hint": "Read the error message, fix the issue, and try again."}
```

Listing 6: Error-as-tool-result pattern

This allows the LLM to read the error, reason about the cause, and take corrective action—such as changing the base branch from `main` to `master`.

5.4 RAG-Based Personalisation (P3)

OpenPA's personalisation system uses Retrieval-Augmented Generation to build a persistent understanding of each user.

5.4.1 Architecture

- **Embedding model**: Ollama [16] running `nomic-embed-text` locally (768-dimensional vectors, no API key required).
- **Vector store**: ChromaDB [7] with persistent storage, cosine similarity, and metadata filtering by `user_id`.
- **Collections**: Two collections—`user_memories` (personality, interests, work, relationships) and `conversation_history` (semantic search over past messages).

5.4.2 Memory Lifecycle

1. **Acquisition**: The LLM is instructed to call `remember_about_user` whenever the user shares personal information. The system prompt provides explicit examples: *"if a user says 'I'm a software developer and I like hiking,' save TWO memories: one for work and one for interests."*
2. **Embedding**: Each memory is embedded via Ollama and stored in ChromaDB with the user ID and category as metadata.
3. **Retrieval**: On each new message, the user's query is embedded and the top-10 most relevant memories are retrieved by cosine similarity, filtered by user ID.
4. **Injection**: Retrieved memories are appended to the system prompt with relevance scores, providing context without loading all memories.

This approach scales to thousands of memories per user—only the relevant subset is retrieved per conversation turn, keeping the context window small.

5.5 Vibe-Coding Workspace (P4)

The workspace system provides a full git-based development environment for the agent to autonomously develop features.

5.5.1 Tool Suite

Fifteen workspace tools cover the complete development workflow:

- **Exploration**: `list_files`, `read_file`, `grep` (regex), `find` (glob), `inspect` (AST-based extraction of classes, functions, signatures, and docstrings)
- **Modification**: `write_file`, `edit_file` (precise find-and-replace), `delete_file`, `install` (auto-detects pip/uv/npm)

- **Verification**: `run` (any shell command—pytest, npm run build, ruff), `check_syntax` (py_compile + ruff for Python, tsc for TypeScript)
- **Git**: `diff`, `commit_push` (with auto-cleanup), `create` (clone + branch), `cleanup`

5.5.2 Adaptive Workflow

The system prompt defines a four-phase adaptive workflow:

1. **Understand**: Clone the repository, explore the file structure, read relevant code, grep for patterns, inspect function signatures.
2. **Research**: Search online documentation for unfamiliar libraries using `web_search` and `scrape_fetch_page`.
3. **Implement**: Install dependencies, write/edit code, check syntax on each changed file.
4. **Verify (loop)**: Run pytest and npm run build. If either fails, read the error output, fix the code, and re-run. Repeat until all checks pass. If stuck after three attempts, report the issue to the user.

The workspace is automatically cleaned up after a successful `commit_push`, and expires after one hour if abandoned.

5.5.3 Code Inspection

The `workspace_inspect` tool uses Python's `ast` module to extract structured information from source files:

```
{"classes": [{"name": "Agent", "bases": ["object"],
  "methods": [{"name": "__init__", "signature": "def __init__(self, provider: LLMProvider, ...)",
               "docstring": "Initialize the agent with a provider and system prompt."}]},
 "functions": [...], "imports": [...], "constants": [...]}
```

Listing 7: AST-based code inspection output

This allows the agent to understand APIs and constructors before using them, rather than reading entire files.

5.6 Self-Evolution: The Assistant That Extends Itself

A distinctive feature of OpenPA is that the vibe-coding system can be directed at the assistant's *own* codebase. Since OpenPA's source code is hosted on GitHub (`maxwellau2/OpenPA`), the same workspace tools that modify external repositories can modify OpenPA itself.

When a user says "add a LinkedIn tool to OpenPA," the agent executes the following self-modification workflow:

1. **Clone self**: `ws_workspace_create(repo="maxwellau2/OpenPA", branch="feature/linkedin-to`
2. **Learn patterns**: Read an existing tool file (e.g., `backend/tools/mastodon.py`) and the tool registry (`backend/tools/registry.py`) to understand the FastMCP tool pattern, credential handling, and namespace mounting convention.
3. **Generate**: Create `backend/tools/linkedin.py` following the established pattern, with appropriate tool definitions, OAuth credential retrieval, and API calls.
4. **Register**: Update `registry.py` to import and mount the new tool server.
5. **Test**: Write unit tests in `backend/tests/test_linkedin.py` and run the full test suite.
6. **Build**: Run `npm run build` on the frontend to ensure no breakage.
7. **Ship**: Commit, push, and create a pull request for human review.

This makes OpenPA a **self-evolving platform**: the set of services it supports is not fixed at development time but can grow through natural language requests. The MCP-based tool architecture ensures that new tools are automatically discoverable by the agent loop once registered, without requiring changes to the agent, the system prompt, or the frontend.

5.6.1 Evolution Triggers

Self-evolution can be triggered through multiple channels:

- **Direct user request**: A user types "add a LinkedIn tool" in the chat interface. The agent enters workspace mode and executes the self-modification workflow immediately.
- **GitHub Issues**: The agent can monitor its own repository for new issues using `github_list_notifications`. When a user (or community contributor) files an issue such as "Feature request: add Notion integration," the agent can read the issue, understand the requirements, and autonomously begin implementation—creating a branch, writing the code, and linking the resulting PR to the original issue.
- **Scheduled evolution**: Using the `scheduler_schedule_task` tool, periodic self-improvement cycles can be scheduled. For example, a nightly job could check for open feature-request issues and attempt to implement the highest-priority one.

5.6.2 Safety Bounds

The self-evolution capability is bounded by multiple safety mechanisms:

- Changes are submitted as **pull requests**, not merged directly—a human reviews before deployment.
- The **full test suite must pass** before pushing; broken code is never shipped.
- The workspace is **cleaned up automatically** after push, preventing resource leakage.
- The agent operates on a **cloned copy** of the repository, not the live deployment, eliminating the risk of corrupting the running instance.

5.7 System Prompt Design

The system prompt (approximately 4,500 words) follows a structured format inspired by Claude Code [3] and Pi [5]:

1. **Identity and scope**: Establishes the agent's role and capabilities.
2. **Action rules**: Always use tools; never apologise for lack of access; parallelise independent tool calls.
3. **Memory rules**: Proactively save user facts; check existing memories before acting.
4. **Tool preference hierarchy**: Explicit mapping of task types to tool categories (e.g., "use workspace tools for repo work, sandbox for one-off scripts").
5. **Output rules**: Be concise; lead with the answer; summarise tool output.
6. **Coding rules**: Don't over-engineer; read before writing; never push broken code.
7. **Workflow templates**: Detailed vibe-coding and self-evolution workflows.
8. **Concrete examples**: 20+ example user queries mapped to expected tool call sequences.

Dynamic context—including the user's email, connected services, and RAG-retrieved memories—is appended after the static rules, forming a "prompt sandwich" where stable instructions bracket variable context.

5.8 Multi-Provider LLM Support (P5)

OpenPA defines an abstract `LLMProvider` interface with a single method:

```
async def chat(self, messages: list[Message],
               tools: list[dict] | None, system: str | None) ->
                   LLMResponse
```

Six providers implement this interface, each converting the unified message and tool format to provider-specific schemas.

OpenPA adopts a **Bring-Your-Own-Key (BYOK)** model: the server stores no LLM provider credentials. Each user configures their own API keys through the settings page, and these are stored per-user in the database. This design has several advantages:

- **Cost transparency**: Users pay their own provider bills directly; the platform operator bears no inference cost.
- **No vendor lock-in**: One user may use Gemini's free tier for casual tasks while another uses Claude Sonnet for complex workflows—on the same OpenPA instance.
- **Privacy**: Users with strict data requirements can use Ollama [16] with locally-hosted models (e.g., Gemma 4, Qwen 3.5, Llama 3.1), ensuring no data leaves their machine.
- **Scalability**: The server-side cost of adding users is near-zero since inference is offloaded to external providers or local hardware.

6 Illustrative Examples

This section demonstrates OpenPA's capabilities through four real-world scenarios, each exercising different aspects of the system.

6.1 Example 1: Daily Briefing (Multi-Service Orchestration)

User: *"Give me a daily briefing."*

Tool filtering: The keywords "daily" and "briefing" activate the *communication* and *productivity* categories, exposing 30 tools (from a total of 88).

Execution: The agent generates a plan with parallel tool calls:

```
Step 1 (parallel):
  - gmail_get_unread()          -> 3 unread emails
  - calendar_list_events()      -> 2 meetings today
  - github_list_notifications() -> 5 PR reviews pending
  - rss_fetch_all_feeds()       -> 12 new articles
Step 2: llm_generate("Summarise these into a concise briefing")
```

Listing 8: Daily briefing plan execution

Result: The user receives a single summary covering email highlights, today's schedule, pending code reviews, and top RSS articles—drawn from four different services in one turn.

6.2 Example 2: Semantic Memory and RAG (Personalisation)

Conversation 1 (previous session):

User: "I'm a software developer who loves hiking and Italian food. My friend Alex is into specialty coffee."

Agent: Calls `remember_about_user` three times:

- Category *work*: "Software developer"
- Category *interests*: "Loves hiking and Italian food"
- Category *relationships*: "Friend Alex is into specialty coffee"

Each memory is embedded via `nomic-embed-text` and stored in ChromaDB.

Conversation 2 (new session, different day):

User: "Who am I?"

RAG retrieval: The query "who am I" is embedded. The top-10 most relevant memories are retrieved by cosine similarity, filtered by user ID. The system prompt is augmented with:

```
## What you know about this user (retrieved by relevance)
  - [work] Software developer (relevance: 0.87)
  - [interests] Loves hiking and Italian food (relevance: 0.82)
  - [relationships] Friend Alex, into specialty coffee (relevance: 0.79)
```

Listing 9: RAG-injected context

Agent: "You're a software developer who loves hiking and Italian food. Your friend Alex is into specialty coffee."

The agent recalls this information *without* accessing the previous conversation—the RAG system retrieves it semantically from the persistent memory store.

6.3 Example 3: Vibe-Coding (Autonomous Code Generation)

User: *"Add a health check endpoint to the REST API that returns the server uptime and version."*

Tool filtering: Keywords "add" and "endpoint" activate *workspace* + *github* + *core* (34 tools). The planner routes this to the adaptive loop.

Execution trace (15 tool calls):

1. `ws_workspace_create(repo="maxwellau2/OpenPA", branch="feature/health-check")`
   → Clones repo, creates branch. Returns workspace ID and default branch ("master").

2. `ws_workspace_list_files(workspace_id, path="backend/services")`
   → Identifies `rest_api.py` as the target file.

3. `ws_workspace_read_file(workspace_id, path="backend/services/rest_api.py")`
   → Reads existing endpoints to understand patterns and conventions.

4. `ws_workspace_grep(workspace_id, pattern="@app.(get|post)", include="*.py")`
   → Finds all route definitions to follow existing style.

5. `ws_workspace_edit_file(workspace_id, path="backend/services/rest_api.py", old_text="...", new_text="...")`
   → Adds `/api/health` endpoint returning uptime and version.

6. `ws_workspace_check_syntax(workspace_id, path="backend/services/rest_api.py")`
   → `py_compile` + `ruff` pass.

7. `ws_workspace_run(workspace_id, "cd backend && python -m pytest tests/ -x -v")`
   → All 32 tests pass.

8. `ws_workspace_run(workspace_id, "cd frontend && npm run build")`
   → Build succeeds.

9. `ws_workspace_commit_push(workspace_id, "feat: add health check endpoint")`
   → Commits, pushes to `feature/health-check`. Workspace auto-cleaned.

10. `github_create_pr(repo="maxwellau2/OpenPA", head="feature/health-check", title="feat Add health check endpoint")`
    → PR created. Base branch auto-detected as "master".

The agent explored the codebase, understood conventions (via grep), made targeted changes, verified syntax, ran the full test suite and frontend build, and submitted a pull request—all autonomously. Had tests failed, the adaptive loop would have read the error output, fixed the code, and re-run the tests.

6.4 Example 4: Cross-Service Workflow (Plan-Then-Execute)

User: *"My friend Alex wants to open a specialty coffee roastery. Do research online to see what the best specialty coffee shops in Singapore and globally are called, pick up the*

trend, and propose 10 names based on that trend. Keep it original and message him on Telegram about it."

Execution plan (5 steps with dependencies):

1. `memory_remember_about_user(content="Friend Alex wants to open specialty coffee roastery", category="relationships")`
2. `web_search(query="best specialty coffee shops Singapore")` → Returns top results.
3. `web_search(query="famous specialty coffee roasters globally")` → Returns global brands.
4. `llm_generate(prompt="Analyse the naming trends... propose 10 original names...")` `depends_on: [2, 3]` → Receives search results as context.
   Generates: Origin Story, Bean Dialect, First Pour, Ember Roast Co., Altitude Coffee, Ground Theory, The Slow Extract, Copper Kettle, Basecamp Beans, Torrefazione.
5. `telegram_send_message(to="Alex", message={step 4 output})` `depends_on: 4` → Sends the generated names via Telegram.

This example demonstrates: memory acquisition (step 1), web research (steps 2–3), text generation with context injection (step 4), and cross-service delivery (step 5)—all from a single natural language request.

7 Evaluation

This section evaluates OpenPA across four dimensions: a qualitative comparison with existing systems, tool integration scope, context-aware tool filtering effectiveness, and multi-provider LLM compatibility.

7.1 Qualitative Comparison with Baselines

Table 3 extends the capability comparison from Section 3 with a qualitative assessment across eight dimensions. Each system is rated on a three-point scale: ✗ (not supported), ∼ (partial), and ✓(fully supported).

Table 3: Qualitative comparison of OpenPA against baseline systems

System	Services	Multi-Step	Code Gen	Self-Evolve	Personalisation	Self-Host	Multi-LLM	Multi-User
Google Assistant	∼[a]	✗	✗	✗	∼[b]	✗	✗	✗
Apple Siri	∼[a]	✗	✗	✗	∼[b]	✗	✗	✗
Amazon Alexa	∼[a]	✗	✗	✗	✗	✗	✗	✗
Claude Code	✗	✓	✓	✗	✗	✗	✗	✗
Codex CLI	✗	✓	✓	✗	✗	✓	✗	✗
Pi Agent	✗	✓	✓	✗	✗	✓	∼[c]	✗
LangChain	✗[d]	✓	✗	✗	∼[e]	✓	✓	✗
AutoGPT	✗[d]	✓	∼	✗	✗	✓	✗	✗
OpenPA	✓	✓	✓	✓	✓	✓	✓	✓

[a]Limited to parent ecosystem services (Google, Apple, Amazon). [b]Explicit preferences only; no semantic retrieval. [c]Supports OpenAI and some local models; no unified provider abstraction. [d]Frameworks requiring custom per-service integration. [e]Supports memory modules but requires developer configuration.

Key observations. Commercial assistants (Google, Siri, Alexa) support a moderate number of services but are constrained to their respective ecosystems and cannot perform multi-step reasoning, code generation, or self-hosting. Coding agents (Claude Code, Codex CLI, Pi) excel at autonomous code generation and multi-step task execution but operate in isolation from broader digital services—they cannot send emails, post to social media, or manage calendars. Agent frameworks (LangChain, AutoGPT) provide composable building blocks but are developer tools, not end-user applications; they require custom integration code for each service and lack built-in OAuth management, multi-tenancy, or chat interfaces.

OpenPA is the only system that simultaneously supports all eight dimensions: 16 integrated services with 88 MCP tools, multi-step orchestration via dual-mode planning, autonomous code generation with test-fix loops, self-evolution through vibe-coding on its own codebase, RAG-based personalisation with persistent semantic memory, full self-hosting capability, 6 LLM providers with BYOK, and per-user multi-tenancy with credential isolation.

7.2 Service Integration Scope

Table 4 quantifies the tool integration across five functional categories. This represents the breadth of the MCP tool registry as implemented.

Table 4: Tool distribution across functional categories

Category	Services	Tools
Communication	Gmail, Discord, Telegram	12
Development	GitHub, Workspace, Sandbox	35
Media & Social	Spotify, Mastodon, YouTube	16
Productivity	Calendar, RSS, Scheduler, Weather	13
Intelligence	Memory, Web Search, Web Scrape	12
Total	**16 services**	**88**

The Development category accounts for the largest share (40%) of tools, reflecting

the vibe-coding system's requirement for fine-grained file operations, git commands, and build/test execution. The Intelligence category—memory, search, and scraping—provides the foundational capabilities that other categories rely on (e.g., web research informing a Telegram message, or memory retrieval personalising a calendar summary).

All 88 tools pass automated validation: each tool is registered with a non-empty description and a typed input schema including the `_user_id` parameter for credential isolation.

7.3 Context-Aware Tool Filtering

A key design contribution is the dynamic tool filtering system that reduces the tool set exposed to the LLM based on task context. Table 5 shows the filtering behaviour for representative query types.

Table 5: Tool filtering: tools exposed per query type (from 88 total)

Query Type	**Categories Activated**	**Tools**	**Reduction**
"Who am I?"	core	8	90.8%
"Play some chill music"	media + core	15	82.8%
"Send Alex a message"	communication + core	20	77.0%
"What's trending on Mastodon?"	social + core	17	80.5%
"Summarise my emails and schedule"	communication + productivity + core	28	67.8%
"Add a feature to the repo"	workspace + github + core	36	58.6%
"Build and test the project"	workspace + github + sandbox + core	43	50.6%
Average		*24*	*72.6%*

On average, the filtering system reduces the tool set by 72.6%, from 88 to approximately 24 tools. For simple queries ("Who am I?"), the reduction is over 90%, exposing only 8 core memory tools. This is critical for smaller local models (7–9B parameters) where large tool definition blocks consume a significant portion of the context window and degrade tool-calling accuracy.

The filtering system includes a safety fallback: if fewer than 3 tools remain after filtering, all 88 tools are exposed to avoid over-constraining the model. During execution, the filter dynamically expands when workspace tools are invoked—adding GitHub and web tools for related operations such as creating pull requests or researching documentation.

7.4 Multi-Provider LLM Compatibility

OpenPA supports six LLM providers through a unified provider abstraction. Table 6 summarises the providers, their available models, and key trade-offs observed during development and testing.

Table 6: LLM provider compatibility and observed characteristics

Provider	Models	Cost	Tool Calling	Notes
Anthropic	Claude Sonnet 4, Haiku 4.5	Pay-per-token	Excellent	Best multi-step reasoning
Google	Gemini 2.5 Flash/Pro, 2.0 Lite	Free tier	Good	Free tier: 15 req/min
OpenAI	GPT-4o, GPT-4o-mini, GPT-3.5	Pay-per-token	Excellent	Widely supported
Groq	Llama 3.3 70B, Mixtral, Gemma	Free tier	Good	Very fast inference
OpenRouter	Gemini Flash, Llama 70B, Qwen	Free tier	Moderate	Free models available
Ollama	Gemma 4, Qwen 3.5, Llama 3.1	Free (local)	Limited	No API key; privacy-first

Observations. Cloud providers (Anthropic, OpenAI) demonstrate the strongest tool-calling accuracy, reliably selecting the correct tool and constructing valid argument schemas in multi-step workflows. Google Gemini and Groq perform well for single-step and moderately complex tasks, with Gemini's free tier making it suitable for casual usage. Local models via Ollama (7–9B parameters) handle simple queries and single-tool invocations but struggle with complex multi-step plans involving 3+ tool calls—the context-aware tool filtering described above is essential for these models.

The BYOK architecture ensures that the platform operator bears no inference cost. Each user configures their own API keys, and models can be switched per-conversation—allowing a user to use Gemini's free tier for casual tasks and Claude Sonnet for complex coding workflows on the same OpenPA instance.

7.5 Agent Execution Characteristics

Table 7 summarises the key runtime parameters governing agent behaviour.

Table 7: Agent loop runtime configuration

Parameter	Value	Purpose
Max iterations (standard)	30	Prevents infinite loops for general tasks
Max iterations (workspace)	80	Allows extended test-fix cycles for coding
Command timeout	120 s	Bounds long-running build/test processes
Max command output	15 000 chars	Prevents context overflow from verbose logs
Workspace expiry	3 600 s	Auto-cleans abandoned workspaces
RAG retrieval count	10 memories	Top-k relevant memories per query
Progress reminder interval	Every 3 iters	Prevents model drift in long sessions
LLM max output tokens	16 384	Per-response generation limit

The workspace iteration limit (80) is significantly higher than the standard limit (30) because vibe-coding tasks typically involve multiple phases—exploration, implementation, testing, and fixing—each requiring several tool calls. A single "add a feature" request commonly requires 10–20 tool invocations (clone, read files, grep for patterns, edit code, check syntax, run tests, fix failures, commit, push, create PR). The progress reminder injection every three iterations addresses the known issue of model drift in long multi-turn conversations, where the LLM may lose sight of the original objective.

8 Conclusion

8.1 Summary

This project presented OpenPA, an open-source Personal Assistant-as-a-Service platform that unifies 16 digital services through a single LLM-powered chat interface. The system addresses five key challenges: multi-service orchestration via MCP-based tool architecture (88 tools), adaptive agent execution with dual-mode planning, RAG-based personalisation using locally-hosted embeddings, autonomous code generation through a workspace-based development environment, and multi-provider LLM support with context-aware tool filtering.

The design draws inspiration from multiple sources: the MCP tool protocol and streaming execution patterns from Claude Code [3]; the minimal tool philosophy and errors-as-results pattern from the Pi coding agent [5]; the terminal-based coding paradigm from OpenAI Codex CLI [4]; and the need for a unified digital life manager from the author's daily experience of context-switching between fragmented services.

8.2 Limitations

Several limitations should be acknowledged:

1. **Database scalability**: SQLite limits horizontal scaling. A multi-instance deployment would require migration to PostgreSQL or a similar production database.

2. **Local model quality**: While tool filtering significantly improves small model performance (reducing the tool set from 88 to 8–36 per query), local models (7–9B parameters) still struggle with complex multi-step tool-calling compared to cloud models such as Claude Sonnet or Gemini Flash.

3. **Security hardening**: The system stores OAuth tokens and API keys in SQLite without encryption at rest. A production deployment would require a secrets manager or encrypted storage.

4. **Test coverage**: While the test suite covers core functionality (32 tests across authentication, memory, RSS, web search, and tool registration), integration tests for the full agent loop and end-to-end service workflows are not yet implemented.

8.3 Future Work

Several directions for future development are identified:

1. **Kubernetes deployment**: While Docker Compose is provided for local and single-node deployment, Kubernetes manifests with horizontal pod autoscaling would enable production-grade cloud deployment.

2. **PostgreSQL migration**: Replace SQLite with PostgreSQL for multi-instance scaling, connection pooling, and row-level security for multi-tenant isolation.

3. **Fine-tuned tool-calling models**: Fine-tune small open-source models (Qwen, Gemma) specifically on OpenPA's tool schemas to improve local model tool-calling accuracy.

4. **Multi-agent collaboration**: Enable multiple agent instances to collaborate on complex tasks—for example, a "research agent" gathering information while a "coding agent" implements changes, coordinated by an orchestrator.

5. **Voice interface**: Add speech-to-text and text-to-speech capabilities for hands-free interaction, building on the existing SSE streaming architecture.

6. **Plugin marketplace**: Leverage the MCP-based architecture to allow users to install third-party tool packages, extending the system's capabilities without modifying the core codebase.

7. **The RALF Loop—fully autonomous self-evolution**: The most ambitious future direction is a closed-loop autonomous evolution cycle we term **RALF** (*Research, Assess, Launch, Feedback*):

   a) **Research**: The agent periodically scrapes the internet—tech blogs, Hacker News, Product Hunt, GitHub trending, Reddit—for emerging tools, APIs, and user-requested integrations. It also monitors its own GitHub issues and community feature requests.

   b) **Assess**: Collected ideas are ranked by a scoring function that considers implementation feasibility (estimated complexity based on similar existing tools), user demand (frequency of related requests in conversation history and GitHub issues), and strategic value (does this fill a gap in the current service coverage?).

   c) **Launch**: The highest-ranked idea is automatically implemented via the vibe-coding workspace: the agent clones itself, reads existing tool patterns, generates the new integration, writes tests, verifies the build, and submits a pull request.

   d) **Feedback**: After deployment, the agent monitors usage of the new tool (call frequency, error rates, user satisfaction from conversation context). Underperforming tools are flagged for improvement in the next cycle; successful patterns inform future assessments.

   The RALF loop would transform OpenPA from a system that evolves *on demand* (when a user asks) to one that evolves *proactively*—continuously discovering, prioritising, implementing, and refining new capabilities without human initiation. This represents a step toward truly autonomous software systems that maintain and improve themselves over time.

8.4 Closing Remarks

OpenPA demonstrates that an open-source, self-hosted personal assistant can achieve service breadth and agentic capabilities competitive with commercial offerings, while providing LLM provider flexibility, persistent personalisation, and autonomous code generation. By building on open standards (MCP) and open-source components (FastMCP, ChromaDB, Ollama), the system ensures that users retain full control over their data, their choice of AI model, and the evolution of their assistant.

References

[1] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023.

[2] Timo Schick et al. "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *Advances in Neural Information Processing Systems* 36 (2023).

[3] Anthropic. *Claude Code: Agentic Coding Tool*. https://docs.anthropic.com/en/docs/claude-code. Terminal-based AI coding agent with tool use, context management, and streaming execution. 2025.

[4] OpenAI. *Codex CLI: Open-Source Coding Agent*. https://github.com/openai/codex. Terminal-based code generation and execution agent. 2025.

[5] Mario Zechner. *Pi: A Minimal Terminal Coding Harness*. https://github.com/badlogic/pi-mono. Open-source coding agent with extensible tool system and prompt template architecture. 2025.

[6] Anthropic. *Model Context Protocol*. https://modelcontextprotocol.io. Open standard for connecting AI assistants to external tools and data sources. 2024.

[7] Chroma. *Chroma: The Open-Source Embedding Database*. https://www.trychroma.com. Open-source vector database for AI applications. 2024.

[8] Harrison Chase. "LangChain". In: (2022). Framework for developing applications powered by language models.

[9] Significant Gravitas. *Auto-GPT: An Autonomous GPT-4 Experiment*. https://github.com/Significant-Gravitas/AutoGPT. Autonomous AI agent framework. 2023.

[10] GitHub. *GitHub Copilot*. https://github.com/features/copilot. AI-powered code completion and suggestion tool. 2024.

[11] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[12] Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.

[13] Vercel. *Next.js: The React Framework for the Web*. https://nextjs.org. Full-stack React framework with server-side rendering. 2024.

[14] Sebastián Ramírez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. https://fastapi.tiangolo.com. High-performance Python web framework based on standard type hints. 2024.

[15] FastMCP Contributors. *FastMCP: Python MCP Server Framework*. https://github.com/jlowin/fastmcp. High-performance Python framework for building MCP-compliant tool servers. 2024.

[16] Ollama. *Ollama: Run Large Language Models Locally*. https://ollama.com. Local LLM inference platform supporting multiple open-source models. 2024.